

Can LLMs write OData filters that improve search?

Learnings from building LLM-generated OData filters for Azure AI Search Knowledge Bases

Natural-language search requests often contain two kinds of intent: what the user is asking for, and which documents should be considered as possible answers. Retrieval and ranking handle the first. Structured filters can enforce the second.

The difficulty is that users rarely express those constraints using the exact fields and values defined in a search index. They use aliases, comparisons, exclusions, and ordinary phrases whose meaning depends on how an index represents its content. A system must decide not only how to translate that language into a filter, but whether the user query justifies filtering at all.

We investigated whether language models could turn ordinary search requests into query-time filters that improve search quality for customers. This post aims to share what we learnt.

What does a search filter do? Why ask an LLM to write one?

Azure AI Search ranks documents by how well they match a query. A filter plays a different role: it decides which documents are eligible to be ranked. In Azure AI Search, these filters use OData, an expression syntax for combining conditions over structured fields such as product, market, year, document type, or configuration.

A query such as “setup instructions for the X100 sold in the US” might yield:

```
product eq 'X100' and region eq 'US'
```

Unlike a ranking signal, a filter acts as a gate: a document either satisfies it or it does not. This can narrow the search space and potentially make retrieval more efficient, but an unsupported condition can also exclude relevant documents entirely.

The challenge is translating ordinary language into the right gate. Users rarely ask for `product eq 'X100'`. They use aliases, omit field names, combine constraints with topical language, and sometimes mention entities that should not restrict the result set.

Static parsers can handle anticipated patterns, but every schema, value vocabulary, and phrasing variation expands the rule surface.

An LLM, instead, can interpret each request against the fields, descriptions, representative values, syntax guidance, and optional field instructions available at query time. As the request changes, the generated constraint can change with it, without requiring a separate library of filter templates.

From a live query to retrieval

The model decides whether structured scope will help before search runs.

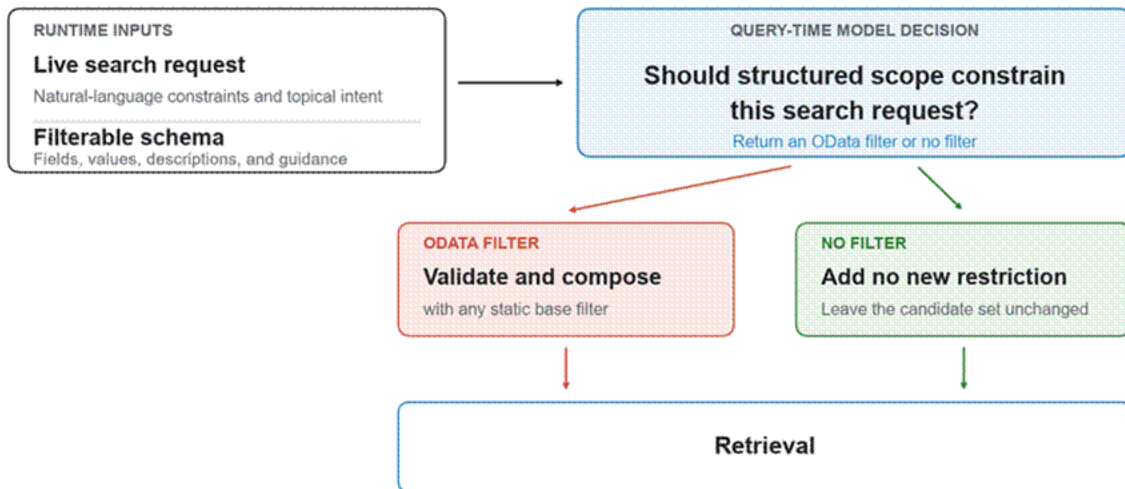


Figure 1. Query-time filter generation decides whether to constrain retrieval or leave the candidate set unchanged.

What makes generating filters hard?

To see why this problem is harder than it sounds, consider a seemingly straightforward query.

Show me cars that don't have all wheel drive.

The model represented it as:

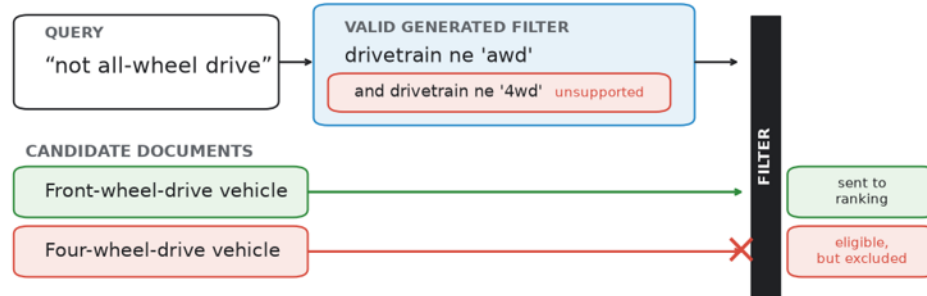
```
drivetrain ne 'awd' and drivetrain ne '4wd'
```

At first glance, the second condition might look like a harmless way to cover two names for all-wheel drive. In this index, however, awd and 4wd are distinct values. The user excluded awd, not 4wd, so the additional clause removes eligible four-wheel-drive configurations before ranking can consider them.

The expression is valid, and every field and value exists in the schema, but it changes the meaning of the request. We call this a **phantom exclusion**: a plausible filter that silently removes evidence. The problem was therefore not simply whether an LLM could produce valid OData, but whether it could decide how far a filter should go, and when no filter was safer.

A filter is an irreversible gate

A plausible extra restriction can remove an eligible result before ranking begins.



Ranking cannot recover what filtering removes.

The syntax is valid; the unsupported scope is the error.

Figure 2. Simplified illustration of a phantom exclusion: the expression is valid, but its scope is not.

What makes generating filters useful?

The example above shows that a filter can be syntactically valid and still exclude relevant documents. We therefore treated useful filter generation as more than producing a well-formed OData expression. A useful system had to:

- **Preserve intent:** express the requested metadata constraints and no others.
- **Abstain when appropriate:** return no filter when the query does not justify one.
- **Remain stable:** make the same semantic decision across repeated generations.
- **Improve retrieval:** produce better ranked results, not merely resemble a reference expression.

These requirements also changed how we evaluated mistakes. An omitted restriction leaves ranking with more documents to consider, which we call **under-filtering**. An unsupported restriction can remove the right document before ranking begins, which we call **over-filtering**. This can directly hurt recall. We tracked them separately.

Our question, therefore, evolved from “Can the model write a valid filter?” to “Can it make a reliable filtering decision that improves retrieval without excluding evidence the query still permits?”

Building a benchmark for filter generation

Evaluating filter generation is difficult for the same reason generation is difficult: the important question is not whether an expression “looks right”, but what it does to the candidate set. A valid expression can change the user’s intent. Different strings can describe the same eligible documents. And a plausible reference filter may still fail to improve retrieval.

We built five purpose-built evaluation environments to make those decisions observable. Each environment paired natural-language queries with an oracle filtering decision over a live Azure AI Search index: either the expected OData filter or an explicit no-filter decision when filtering would be unsafe or unnecessary. Each oracle decision was reviewed against the live index so the benchmark reflected constraints the schema could actually support, not just constraints that sounded plausible from the query. The datasets were built specifically for this task from a mix of real corpora, product documents, and deterministic synthetic corpora. Rather than create five copies of one task, we used the environments to stress different combinations of schema mapping, multilingual language, abstention, Boolean composition, typed fields, and the relationship between metadata and document text.

The broader audited suite covered more OData patterns than we analyze here. For this post, we focused on the high-confidence core of the task: equality, inequality, and Boolean combinations over supported filterable fields. This surface covered most eligibility constraints in the evaluation and gave us the clearest evidence of reliable behavior.

Environment	How it was constructed and what it tests	In-scope queries	Filter / null
Product support	Real product documents with authored multilingual queries. Tests aliases, encoded values, and schema mapping.	111	85 / 26
Device manuals	Synthetic support documents and queries. Tests single-field mapping and whether the model abstains when metadata should not constrain retrieval.	81	32 / 49
Company filings	Real filing documents with authored queries. Tests whether companies, filing types, sectors, and periods describe scope or merely the topic.	159	104 / 55
Industry documents	Multilingual, multi-industry documents with a broad purpose-built schema. Tests domain vocabulary and scope across 25 industry-specific fields.	394	336 / 58
Vehicle configurations	Synthetic configuration records and compositional queries. Tests explicit Boolean logic across one to five predicates.	300	240 / 60

Evaluation scope. The broader audited suite contained 1,174 queries. For this analysis, we focus on the 1,045-query subset aligned with the first release surface: `eq`, `ne`, `and`, and `or` over supported filterable fields, plus explicit no-filter cases. This keeps the reported results tied to the operator surface we are shipping first.

We evaluated the resulting decisions using five metrics:

- **Accepted decision rate:** the percentage of outputs accepted as equivalent to or defensibly better than the oracle filter, including correct no-filter decisions. These were rated by an LLM judge.
- **Harmful over-filtering rate:** the percentage of outputs that add an unsupported or incorrect restriction capable of excluding valid results.
- **Under-filtering rate:** the percentage that omit a required predicate or return no filter when filtering is warranted.
- **Semantic repeatability:** the percentage of repeated-generation pairs that express the same filtering decision.
- **Retrieval lift:** the change in retrieval metrics when retrieval uses the generated filter instead of no filter.

With the evaluation framework in place, we turned to the practical questions it was designed to answer: which configuration to use, how reliably it made the same decision, and whether those decisions improved retrieval.

How model size and reasoning effort change performance

We tested 16 model-and-reasoning configurations chosen to represent the model families and reasoning settings commonly considered for Azure AI Search Knowledge Base workloads. The sweep included full-size models for the primary deployment comparison, plus mini and nano variants as lower-cost boundary cases.

The sweep produced a clear leading group, but no configuration was best along every dimension. Frontier GPT-5.4 and GPT-5.5 configurations achieved similar accepted decision rates while differing in harmful over-filtering, repeatability, and performance across environments.

GPT-5.4 with low reasoning effort was accepted on 1,000 of 1,045 queries, or 95.7%, while GPT-5.5 without reasoning reached 95.9%, with 1,002 accepted decisions. That two-query advantage concealed a larger difference in harmful over-filtering: GPT-5.4 with low reasoning effort added three unsupported restrictions capable of excluding valid results, compared with nine for GPT-5.5 without reasoning. Judged only by aggregate acceptance, the configuration with more harmful failures would have appeared better.

Similar acceptance rates concealed different failure profiles

Full-size models only · higher acceptance and lower harmful over-filtering are preferred.

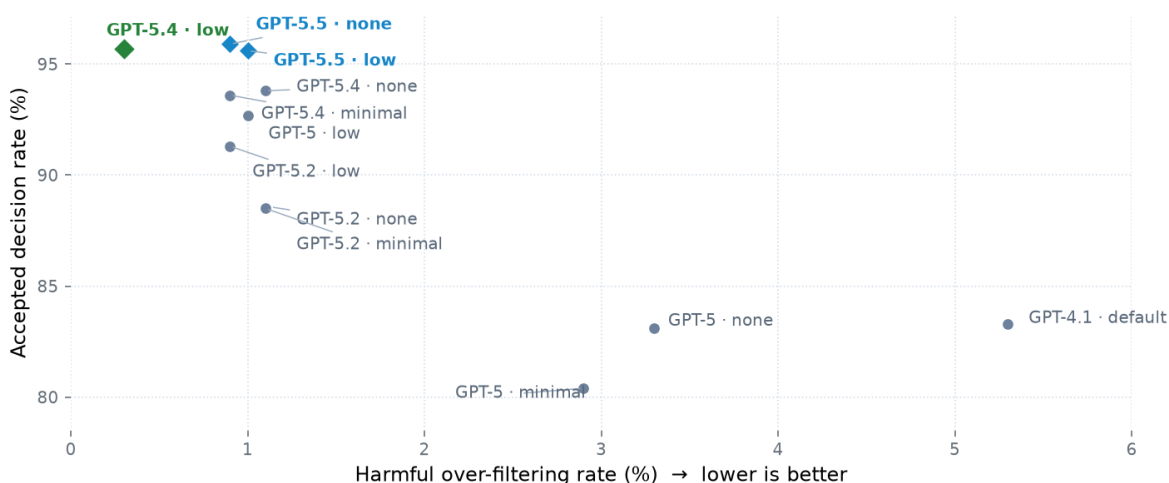


Figure 3. GPT-5.4 with low reasoning provided the strongest balance of acceptance and harmful over-filtering.

The lower-cost variants also revealed how failure behavior changed as model capability decreased. Some smaller configurations did not simply miss constraints or abstain more often; they produced substantially more harmful over-filtering. GPT-5.4 mini without reasoning, for example, had a 10.0% harmful over-filtering rate overall, rising above 20% on device manuals. Model capability therefore affected both how often a decision was accepted and the direction and potential consequence of its failures.

Reasoning effort changed behavior rather than simply improving it. Moving from None to Low reasoning helped GPT-5 recover more valid filters, raising accepted decisions from 83.1% to 92.7%

and reducing harmful over-filtering from 3.3% to 1.0%. But the same pattern did not scale uniformly across stronger models or harder workloads. GPT-5.5 was effectively unchanged, and in company-filings queries, where the hardest question was often whether a company name described the comparison topic or the document scope, more reasoning did not resolve the ambiguity. Filter generation rewards restraint as much as inference: the model must know when not to turn a phrase into a constraint. The practical lesson is that reasoning effort should be evaluated as a behavioral setting, not assumed to be a universal quality dial.

GPT-5.4 with low reasoning ultimately provided the strongest overall balance for an unknown query mix. More broadly, configuration quality could not be reduced to one acceptance score, one model size, or one reasoning setting.

How consistently do LLMs output the same filters?

To measure stability separately from correctness, we selected 140 queries and generated ten independent outputs for each. Semantic repeatability measured how often two outputs for the same query made the same filtering decision, even when expressed differently. Ten generations produced 45 pairs per query and 6,300 pairs for each model configuration.

Across the 16 tested configurations, accepted decision rate and semantic repeatability were strongly associated. Configurations that made better filtering decisions generally also made them more consistently.

GPT-5.5 without reasoning reached 98.03% semantic repeatability, with all ten generations agreeing on 133 of 140 queries. GPT-5.4 with low reasoning reached 97.62%, with 131 unanimous queries. Neither configuration produced a structurally invalid output.

Better filtering decisions were generally more repeatable

Accepted decision rate and semantic repeatability across all 16 tested configurations.

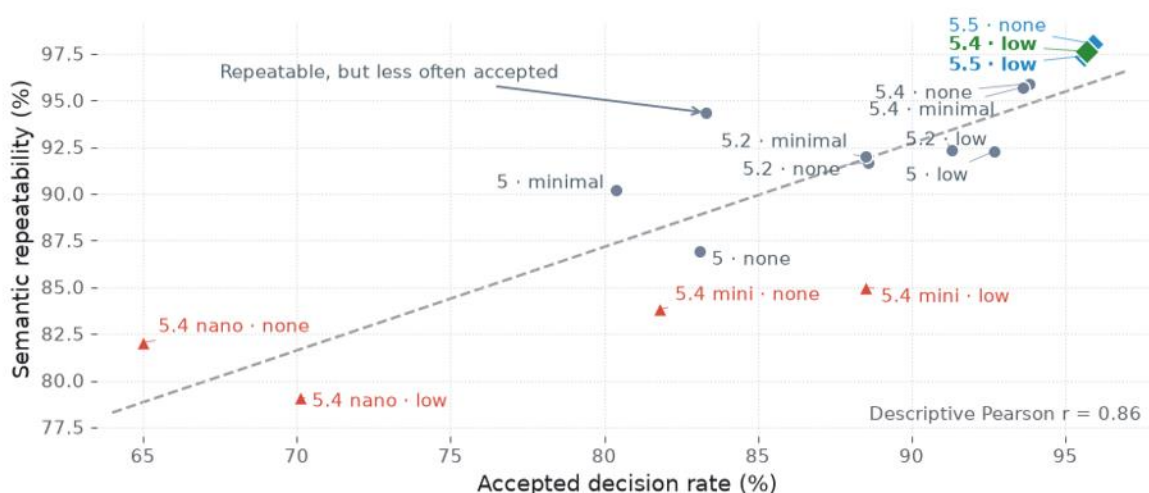


Figure 4. Accepted decision rate and semantic repeatability generally moved together across the 16 tested configurations, but high consistency did not guarantee correctness.

The smaller models were less stable as well as less accurate. GPT-5.4 mini and nano configurations ranged from 79.08% to 85.00% semantic repeatability, with as few as 73 unanimous queries. Their failures were not only more frequent; the models were also less consistent about which interpretation of the query to apply. In other words, lower capability did not just change which filters were produced; it made the boundary between filtering and abstaining less stable.

The relationship between correctness and stability was not absolute. GPT-4.1 reached 94.35% semantic repeatability despite an accepted decision rate of 83.3%. That exception matters: a model can consistently reproduce the same wrong filter. Repeatability therefore complements correctness rather than replacing it.

The clearest variation cases were not syntax failures; they were disagreements about whether a phrase belonged in the filter at all. For example, for “Which medications increase the risk of falls in older adults, and how should medication-related falls be analyzed?”, eight runs returned no filter, while two emitted `therapeutic_area eq 'geriatrics'`. The filtered version looks reasonable, but it assumes that every useful answer must come from documents explicitly tagged as geriatrics. In this case, “older adults” describes the clinical topic being asked about, not a reliable eligibility rule for the document set. That is exactly the failure mode that matters in retrieval: a plausible-sounding filter can silently exclude useful evidence.

Similar disagreements appeared around aliases, negation, optional predicates, multilingual wording, and scope. The strongest configurations were highly stable overall, but the remaining variation showed where the model’s policy was still unresolved: when should a phrase constrain the eligible document set, and when should it remain part of the question being answered?

How much do the filters really help search?

So far, we have measured whether the model made the right filtering decision: did it apply the metadata constraints the query justified, abstain when filtering was unsafe, and do so consistently? The next question was whether those decisions actually improved retrieval.

We use NDCG@5 as the headline retrieval metric because it captures both relevance and rank: a relevant result counts more if it appears near the top of the first five results. We also checked P@5, the share of the top five results that were relevant, and Recall@5, the share of known relevant results recovered in the top five. Together, these metrics show whether filtering improved ranking order, top-five precision, or relevant-result coverage.

Filtering helped most when metadata supplied missing scope

Generated filters helped most when metadata carried scope that text ranking could not reliably infer.

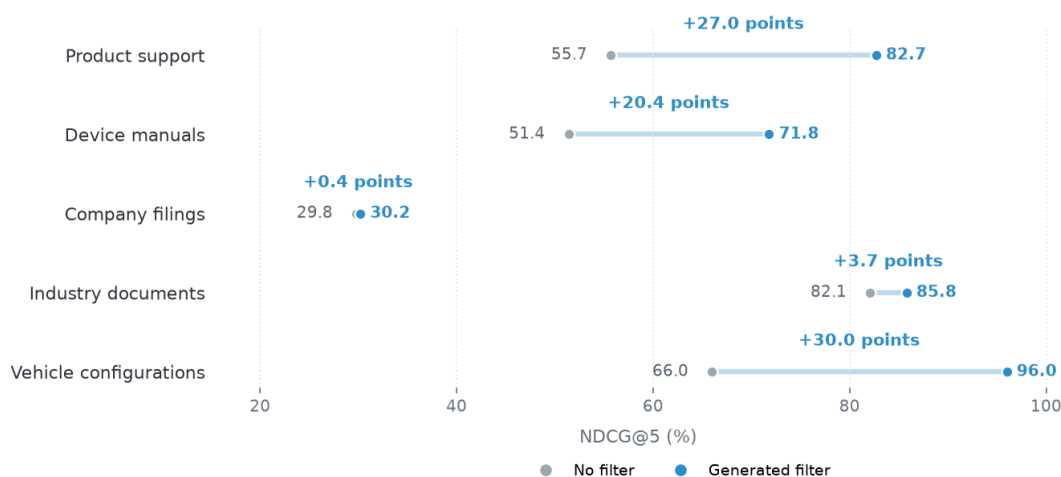


Figure 5. Generated filters improved NDCG@5 in all five environments, with gains ranging from 0.4 to 30.0 points.

The retrieval gains were positive across all five environments, but they were not uniform. The largest gains appeared where many documents could sound similar in prose while differing in structured metadata. Vehicle configurations improved from 66.0% to 96.0% NDCG@5, a 30.0-point gain. Product support improved from 55.7% to 82.7%, and device manuals improved from 51.4% to 71.8%. In these settings, the generated filter was not just making the query more specific. It changed the candidate set before ranking began, deciding which documents were eligible to be ranked.

That distinction matters because metadata can carry scope that text ranking sees only weakly. Vehicle records may share nearly identical descriptions while differing on model line, drivetrain, equipment, or warranty fields. Product and device documentation can differ by product family, access level, manual type, or support category even when the surrounding prose is similar. When the model mapped those cues into filters, retrieval improved because the ranker no longer had to separate lookalike documents using content alone.

The flatter environments are just as informative. Company filings improved only 0.4 points on NDCG@5, with P@5 essentially unchanged. That is plausible because company names, filing periods, and financial terms are often already visible in the text being retrieved. Industry documents showed a more modest gain of 3.7 points, suggesting that filters helped on some scoped queries but were not the dominant retrieval signal.

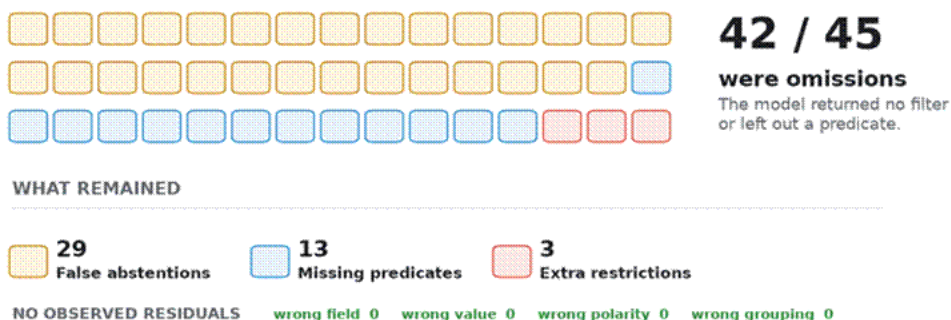
The behavioral takeaway is that generated filters are most valuable when the user's request contains eligibility information that is reliable in metadata but weak or ambiguous in text. When that scope is already visible to lexical or semantic ranking, the filter has less work to do.

Inspecting the anatomy of filtering errors

We next inspected the 45 outputs from GPT-5.4 with low reasoning that were not accepted by our LLM judge evaluation. The interesting question was not just how often the model failed, but which direction it failed in: did it leave too many documents for retrieval to rank, or did it exclude documents the user still needed.

Most remaining misses broadened the candidate set

After 1,000 of 1,045 outputs were accepted, 45 cases remained.



The harder decision was what to filter, not how to write the filter.

Figure 6. Forty-two of 45 remaining misses were conservative omissions: false abstentions or missing predicates.

The model returned no filter when a hard filter was required in 29 cases and omitted at least one required predicate in 13 more. It added an unnecessary restriction only three times. We found no residual cases where the model used the wrong field, wrong value, wrong polarity, or malformed Boolean grouping.

That distribution matters because filter errors are asymmetric. A false abstention or missing predicate leaves too many documents for retrieval to rank. It weakens the benefit of filtering, but usually preserves eligible evidence. A harmful extra restriction is more dangerous: it can remove the right document before ranking ever sees it. The best configuration’s remaining errors were therefore mostly conservative, but not harmless.

The pattern points to a grounding problem rather than a syntax problem. In a French accounting query asking about an accounting standard applicable internationally or in China, the model generated `jurisdiction eq 'Global'` or `jurisdiction eq 'CN'` but omitted `document_type eq 'standard'`. It understood part of the scope, but failed to bind “standard” to the schema field that made the query narrower and more useful.

The three harmful over-filtering cases showed the opposite failure. For “How should Apeos, Frontier, INSTAX, ScanSnap, and X devices be handled safely?”, the correct output was no filter, but the model emitted a device-family filter covering printers, cameras, scanners, and related devices. The output looked plausible, but it crossed the line from interpreting the query into adding product knowledge that was not safely grounded in the schema. That kind of mistake is exactly why over-filtering is more risky than omission.

Taken together, the residual errors suggest that the model had largely learned the OData language and the basic discipline of not inventing fields or values. In other words, the last mile was not writing valid filters; it was knowing when language was grounded enough to become a hard retrieval constraint.

This matched the broader difficulty pattern. We expected long filters and nested Boolean structure to be the hardest cases, but the model handled explicit structure well once the intended constraints were clear. For GPT-5.4 with low reasoning, accepted decisions fell from 99.3% on explicit filtering decisions to 96.0% where the model had to infer a constraint and 89.7% where it had to judge whether or how far to filter. Performance declined as more of the filtering decision was left to interpretation. One inferred predicate was sometimes harder than five explicit ones.

Could targeted guidance mitigate grounding errors?

The error analysis pointed to a mitigation strategy for some of the errors. Many misses were not caused by invalid OData; they came from index-specific meaning the model had to infer. A product nickname might map to a stored model family. A word like “restricted” might imply an access-level value. If the model lacked that local context, it sometimes abstained, missed a predicate, or added a plausible but unsupported restriction. Could we mitigate these errors if we just provided this missing context?

We tested this with FilterInstructions: field-level guidance that explains how a field should be used. The instructions translated the schema’s local vocabulary for the model and clarified when a phrase was grounded enough to become a filter.

FilterInstructions supplied the missing grounding

Field guidance turned local schema meaning into a safer filtering decision.

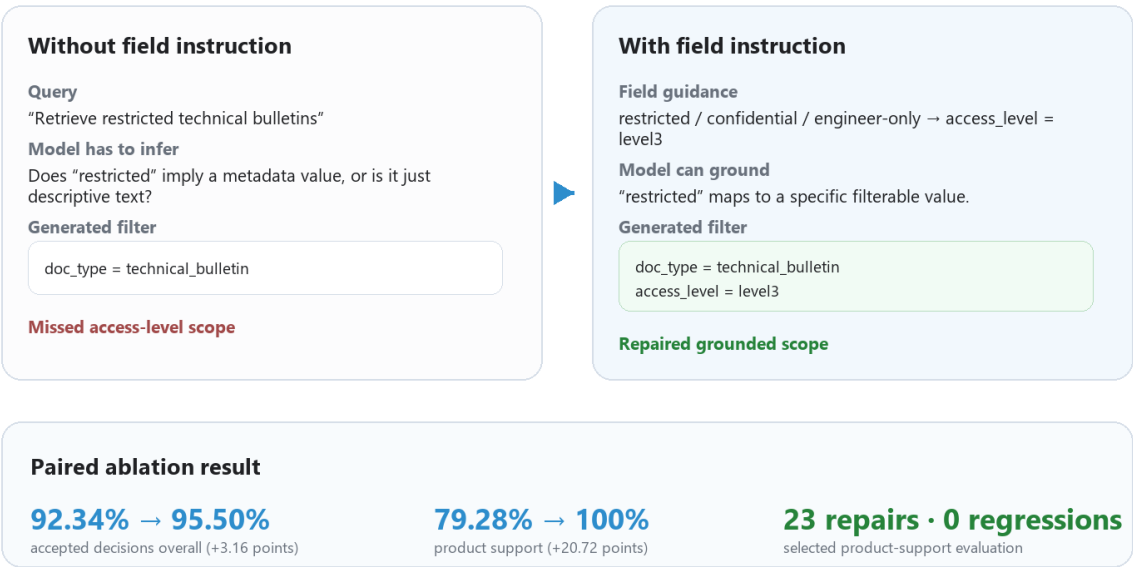


Figure 7. Field guidance turned local schema meaning into a safer filtering decision.

Figure 7 shows this mechanism. Without that guidance, “Retrieve restricted technical bulletins” produced only `doc_type eq 'technical_bulletin'`: the document type was clear, but the access constraint was missed. With field guidance mapping “restricted” to `access_level eq 'level3'`, the model produced the intended filter: `doc_type eq 'technical_bulletin' and access_level eq 'level3'`.

In a controlled ablation over the same 1,045 queries, using GPT-5.4 with low reasoning, we changed only the FilterInstructions. Accepted LLM judge decisions improved by 3.16 percentage points. The largest gain came in product support, where acceptance rose from 79.28% to 100%, repairing 23 failures similar to the example.

The broader lesson is that FilterInstructions are not just extra prompt text. They change the model’s evidentiary boundary: what it can safely treat as grounded in the index, and what it should leave as search intent. The raw schema exposes field names and allowed values, but it does not explain the local meaning of those values or the boundary for using them. Field guidance supplies that missing layer. In this ablation, making that layer explicit gave us a reliable way to improve filter generation: fewer missed constraints, fewer unsupported guesses, and better filters without changing the model.

So, can LLMs write filters that improve search?

As our investigation progressed, the original question became more precise. The hard part was not whether a model could express a filter in OData. It was whether the model could decide when a phrase in the user’s request was grounded enough to become a hard retrieval constraint.

That is the central behavior filter generation asks of a model: drawing the boundary between search intent and document eligibility. Some words in a query describe what the user hopes to find; others specify what must be true of every document before ranking begins. A useful filter depends on putting each phrase on the right side of that line. If the model is too cautious, retrieval gets less help. If it is too assertive, the right evidence can be removed before the ranker ever sees it.

So our answer to the question “Can LLMs write filters that improve search?” is yes, when the task is grounded in metadata the model can see and the feature is designed around safe constraint selection. LLM-generated filters are most useful when metadata captures eligibility information that text ranking alone does not reliably enforce. The goal is not arbitrary OData generation; it is reliable filter generation for the high-confidence cases where filters can improve retrieval before ranking begins. That requires three things working together: a well-chosen model configuration, an operator surface matched to the evaluated task, and schema guidance that tells the model what the index’s fields actually mean.

That is the shape of the feature we are making available in Azure AI Search Knowledge Bases through query hints in Public Preview. For this preview, we focused on the high-confidence operator set used throughout our evaluation: `eq`, `ne`, `and`, and `or` over supported string, numeric, and boolean filterable fields. This is the core surface where generated filters most directly express eligibility constraints before ranking. The feature also lets developers use FilterInstructions to add the local meaning that the raw schema cannot express.

To summarize, our investigation taught us that useful filter generation depends more on judgment than on expressive power. The model does not need to write the most elaborate filter; it needs to decide which constraints are grounded enough to apply before ranking begins. Under these conditions, generated filters can reliably narrow the search space while preserving the evidence the user still needs.